Architettura Computazionale e Visualizzazione delle Mappe di Confidenza $\Delta\chi^2$ 2D

Manuale Tecnico e Guida alle API del Modulo `chi_squared_2d_map`

Laboratorio di Fisica Sperimentale e Computazionale

22 ottobre 2026

Sommario

Questo manuale fornisce la documentazione tecnica del modulo `chi_squared_2d_map.py`, deputato alla rappresentazione grafica rigorosa delle superfici di costo bidimensionali generate da processi di regressione non lineare (ODR Metodo A). Vengono analizzati in dettaglio: la scansione a serpentina profilata con *warm-start* in `Minuit`, l'interpolazione spline cubica bicubitica, l'integrazione nativa con le dimensioni standard `TYPO_CONFIG` di `plot_typo`, la topologia multi-layout della griglia asimmetrica (`colorbar_loc` a destra, in basso o assente), l'allocazione flessibile del box di covarianza e la corretta eliminazione dei tick sui bordi condivisi tramite potatura selettiva.

Indice

1 Introduzione e Integrazione Tipografica **2**

2 Algoritmo di Profilazione e Ricostruzione della Superficie **2**
2.1 Campionamento a Serpentina e Warm-Start Sequenziale 2
2.2 Ricostruzione Continua via Spline Cubica 3

3 Topologia del Canvas e Geometrie Multi-Layout **4**
3.1 Configurazioni Supportate da `colorbar_loc` 4
3.2 Pulizia dei Tick sui Bordi Condivisi 4

4 Contorni di Confidenza e Segnaletica **4**
4.1 Livelli di Confidenza Standard 4
4.2 Leggibilità del Testo con Effetto Contorno (*Halo Stroke*) 4

5 Firma Completa delle API: `chi2_map_pipeline` **5**

6 Flusso Completo di Utilizzo in Laboratorio **6**

1 Introduzione e Integrazione Tipografica

La visualizzazione congiunta dell'incertezza su due parametri (θ_x, θ_y) richiede di superare l'approssimazione parabolica locale HESSE ($V = 2\mathbf{H}^{-1}$). Il modulo `chi_squared_2d_map.py` implementa un'architettura completa che combina:

1. **Scansione di Verosimiglianza Profilata**: a ogni nodo della griglia 2D la coppia (θ_x, θ_y) viene vincolata, ri-minimizzando i restanti parametri liberi con `migrad()`;
2. **Integrazione con** `plot_typo`: importa `format_sci_axis` e il dizionario condiviso `TYPO_CONFIG`, ereditando in modo coerente le dimensioni tipografiche per testi, etichette e lunghezze dei tick centrate `inout`;
3. **Proiezioni Marginali MINOS 1D**: traccia sui bordi i profili unidimensionali calcolati da `mnprofile`, mostrando la perfetta tangenza delle rette di confidenza $1\sigma_{1D}$ ($\Delta\chi^2 = 1.0$) con i contorni 2D;
4. **Layout Asimmetrico Adattivo**: gestione flessibile della barra colori (`'right'`, `'bottom'`, `'none'`) con cuscinetto (*pad*) anti-sovrapposizione.

2 Algoritmo di Profilazione e Ricostruzione della Superficie

2.1 Campionamento a Serpentina e Warm-Start Sequenziale

Dato un modello a p parametri con minimo globale $F_{\min} = \chi^2(\hat{\boldsymbol{\theta}})$, si definisce un intervallo bidimensionale centrato sulle stime di semi-ampiezza N_σ:

$$x \in [\hat{\theta}_x - N_\sigma\sigma_x,\ \hat{\theta}_x + N_\sigma\sigma_x], \qquad y \in [\hat{\theta}_y - N_\sigma\sigma_y,\ \hat{\theta}_y + N_\sigma\sigma_y] \tag{1}$$

dove le incertezze σ_x, σ_y sono estratte considerando il massimo tra gli errori HESSE ed errori asimmetrici MINOS (`merrors`).

La griglia nodale $K \times K$ viene percorsa alternando la direzione di scansione (*boustrophedon*):

```python
for i in range(grid_size):
    j_indices = range(grid_size) if (i % 2 == 0) else reversed(range(grid_size))
    for j in j_indices:
        # Warm-start dallo stato ottimo precedente
        for p in param_names:
            m.values[p] = current_cache[p]

        m.fixed[param_x] = True
        m.fixed[param_y] = True
        m.values[param_x] = X_map[i, j]
        m.values[param_y] = Y_map[i, j]
        m.migrad()

        # Fallback di sicurezza se la discesa non e' valida
        if not m.valid:
            for idx, p in enumerate(param_names):
                m.values[p] = orig_values[idx]
            m.values[param_x] = X_map[i, j]
            m.values[param_y] = Y_map[i, j]
            m.migrad()

        Z_profiled[i, j] = m.fval
        for p in param_names:
            current_cache[p] = m.values[p]
```

Listing 1: Scansione a serpentina con warm-start e ripristino di emergenza.

2.2 Ricostruzione Continua via Spline Cubica

La matrice campionata discretamente viene riferita al minimo assoluto: $\Delta Z_{ij} = Z_{ij} - F_{\min}$. Per prevenire spigolosità poligonali sui contorni chiusi, la griglia viene sovracampionata mediante interpolazione spline cubica (`scipy.ndimage.zoom`) con fattore di ingrandimento Z_f (default: 4):

```
dz_smooth = ndimage.zoom(dz, zoom_factor, order=3)
```

3 Topologia del Canvas e Geometrie Multi-Layout

3.1 Configurazioni Supportate da `colorbar_loc`

Il parametro `colorbar_loc` consente tre partizioni distinte della figura:

1. `"right"` **(Default)**: griglia 2 righe $\times$ 4 colonne con larghezze relative $[3.5, 1.0, 0.1, 0.15]$. La colorbar occupa il canale verticale $[1, 3]$ ed è separata dal profilo laterale destro da uno slot cuscinetto ($cbar\ pad = 0.1$).
2. `"bottom"`: griglia 4 righe $\times$ 2 colonne con altezze relative $[1.0, 3.5, 0.1, 0.15]$. La colorbar viene posizionata orizzontalmente sul riquadro inferiore $[3, 0]$.
3. `"none"`: griglia compatta 2×2 con larghezze $[3.5, 1.0]$ e altezze $[1.0, 3.5]$, eliminando completamente la scala cromatica.

3.2 Pulizia dei Tick sui Bordi Condivisi

L'azzeramento della spaziatura (`spacing = 0.0`) richiede la rimozione delle tacche all'interfaccia tra la mappa 2D e i profili 1D per evitare sovrapposizioni numeriche:

- Soppressione delle label interne: `ax_t.tick_params(axis='x', labelbottom=False)` e `ax_r.tick_params labelleft=False)`.
- **Potatura dei Tick d'Angolo** (`prune="upper"`): per evitare che le cifre estreme dell'asse centrale collidano con le origini dei profili marginali, si applica direttamente sull'asse centrale `ax1` il potatore di Matplotlib:

```
ax1.yaxis.set_major_locator(ticker.MaxNLocator(nbins=6, prune="upper"))
ax1.xaxis.set_major_locator(ticker.MaxNLocator(nbins=6, prune="upper"))

```

  Questa procedura elimina in modo trasparente l'ultimo tick superiore e destro senza alterare la formattazione scientifica prodotta da `format_sci_axis`.

4 Contorni di Confidenza e Segnaletica

4.1 Livelli di Confidenza Standard

Livello Tracciato	Gradi di Libertà	$\Delta\chi^2$	Copertura	Stile Grafico Adottato
$1\sigma_{1D}$ (Marginale)	$\nu = 1$	1.00	68.27%	Tratto-punto nero (`'-.'`, 1.1 pt)
$1\sigma_{2D}$ (Congiunto)	$\nu = 2$	2.30	68.27%	Continuo bianco (`'-'`, 1.6 pt)
$2\sigma_{2D}$ (Congiunto)	$\nu = 2$	6.18	95.45%	Tratteggiato bianco (`'--'`, 1.2 pt)
$3\sigma_{2D}$ (Congiunto)	$\nu = 2$	11.83	99.73%	Punteggiato bianco (`':'`, 0.9 pt)

Tabella 1: Livelli di confidenza standard e stili visivi implementati nel modulo.

4.2 Leggibilità del Testo con Effetto Contorno (*Halo Stroke*)

Le etichette dei contorni vengono rese nitide su qualsiasi gradiente cromatico di sfondo mediante il filtro tipografico `patheffects.withStroke`:

```
halo = [patheffects.withStroke(linewidth=2.5, foreground="white")]
clbl_2d = ax1.clabel(cs_2d, **clabel_opts)
plt.setp(clbl_2d, path_effects=halo)
clbl_1d = ax1.clabel(cs_1d, **clabel_opts)
plt.setp(clbl_1d, path_effects=halo)
```

5 Firma Completa delle API: `chi2_map_pipeline`

```python
def chi2_map_pipeline(
    risultati: Any,
    param_x: str,
    param_y: str,
    param_info: Optional[Dict[str, Dict[str, str]]] = None,
    n_sigma: float = 3.5,
    grid_size: int = 35,
    zoom_factor: int = 4,
    show_clabels: bool = True,
    manual_clabels: Union[bool, List[Tuple[float, float]]] = False,
    clabel_fontsize: Optional[float] = None,
    stats_box_loc: str = "inside_map",  # 'inside_map', 'top', o 'custom'
    stats_box_pos: Optional[Tuple[float, float]] = None,
    show_stats_box: bool = True,
    spacing: float = 0.0,
    cmap: str = "viridis_r",
    figsize: Optional[Tuple[float, float]] = None,
    save_path: Optional[Union[str, Path]] = None,
    show: bool = True,
    colorbar_loc: str = "right",        # 'right', 'bottom', oppure 'none'
) -> Tuple[plt.Figure, Tuple[plt.Axes, plt.Axes, plt.Axes]]:
```

Listing 2: Firma aggiornata della funzione `chi2_map_pipeline`.

Parametro	**Tipo / Default**	**Descrizione e Funzione Operativa**
`risultati`	`RisultatiFit`	Struttura contenente l'istanza ottimizzata `minuit_obj`.
`param_x`, `param_y`	`str`	Nomi delle due variabili di scansione (es. `"m"`, `"q"`).
`param_info`	`dict` / `None`	Metadati dimensionali: label LaTeX (`'label'`) e unità (`'unit'`).
`n_sigma`	`float = 3.5`	Semi-ampiezza della regione di scansione in deviazioni standard.
`grid_size`	`int = 35`	Risoluzione della griglia discreta $K \times K$ prima della spline.
`zoom_factor`	`int = 4`	Moltiplicatore di sovracampionamento spline cubica ($K \cdot Z_f$ nodi).
`show_clabels`	`bool = True`	Abilita le etichette numeriche sui contorni σ_{2D} e $1\sigma_{1D}$.
`manual_clabels`	`bool / list`	Coordinate manuali per il piazzamento delle etichette dei contorni.
`clabel_fontsize`	`float / None`	Dimensione font delle label (se `None`, usa `TYPO_CONFIG["legendsize"]`).
`stats_box_loc`	`str = "inside_map"`	Collocazione del box statistico: `'inside_map'`, `'top'`, `'custom'`.
`stats_box_pos`	`tuple / None`	Coordinate (x, y) in coordinate figura per `stats_box_loc="custom"`.
`spacing`	`float = 0.0`	Spaziatura (`wspace, hspace`) tra i pannelli accostati.
`figsize`	`tuple / None`	Dimensioni della figura in pollici (adattate automaticamente al layout se `None`).
`colorbar_loc`	`str = "right"`	Posizionamento della scala colori: `'right'`, `'bottom'` o `'none'`.
`save_path`	`str / Path / None`	Percorso su disco per il salvataggio vettoriale ad alta definizione.

Tabella 2: Dizionario dei parametri di ingresso della funzione `chi2_map_pipeline`.

6 Flusso Completo di Utilizzo in Laboratorio

```python
import numpy as np
from pathlib import Path
from chi_squared_fit import FitPipeline
from chi_squared_2d_map import chi2_map_pipeline

def modello_lineare(x, m, q):
    return m * x + q

rng = np.random.default_rng(123)
m_vero, q_vero = 2.45, 1.10
n_punti = 35

x_pulito = np.linspace(1.0, 10.0, n_punti)
y_pulito = modello_lineare(x_pulito, m_vero, q_vero)

err_x = np.full(n_punti, 0.20)
err_y = np.full(n_punti, 0.45)

x_misurato = x_pulito + rng.normal(0, err_x, n_punti)
y_misurato = y_pulito + rng.normal(0, err_y, n_punti)

pipeline = FitPipeline(
    model=modello_lineare,
    param_names=["m", "q"],
    p0=[1.0, 0.0],
    run_minos=True,
    verbose=True,
)

risultati_fit = pipeline.fit(
    x=x_misurato,
    y=y_misurato,
    err_x=err_x,
    err_y=err_y,
    odr_lin_Newton=True,
)

info_parametri = {
    "m": {"label": r"$m$", "unit": r"V/s"},
    "q": {"label": r"$q$", "unit": r"V"},
}

fig, (ax_main, ax_top, ax_right) = chi2_map_pipeline(
    risultati=risultati_fit,
    param_x="m",
    param_y="q",
    param_info=info_parametri,
    figsize=(7.5, 6.5),
    stats_box_loc="custom",
    stats_box_pos=(0.725, 0.85),
    show=True,
)
```

Listing 3: Script end-to-end: regressione ortogonale e visualizzazione mappa con box custom.